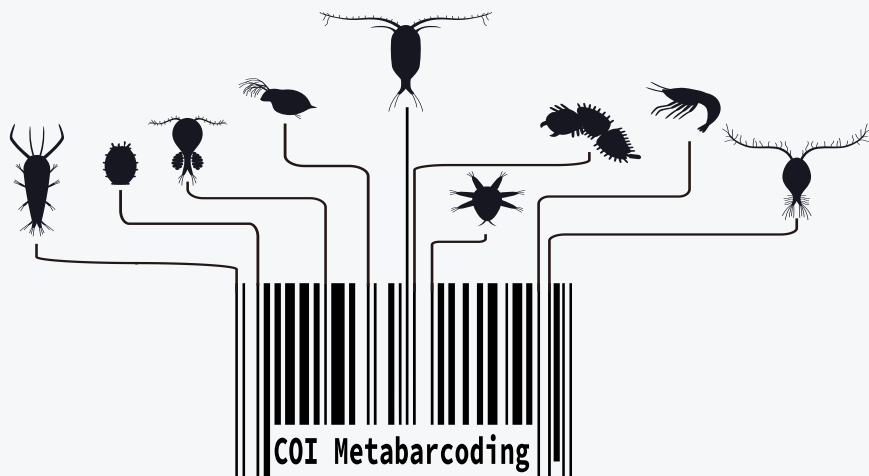


Day1: NGS raw data → ASV

Marine eDNA Metabarcoding Analysis Using R

Jeju ARMS Case Study

2026.08.31



Song, Chi-une

Postdoctoral Researcher
Department of Life Science
Chung-Ang University

Overview — workshop workflow

From a marine sample to ecological interpretation

01

Sampling & DNA Preparation

Collect and prepare the biological material

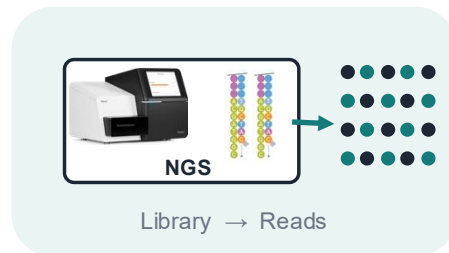


- 1 Marine sample collection
- 2 DNA extraction
- 3 COI barcode amplification

02

NGS Sequencing

Convert amplified DNA into sequencing reads



- 1 Library preparation
- 2 Illumina sequencing
- 3 Raw sequencing output

03

Bioinformatics Processing

Turn raw reads into reliable sequence variants



- 1 FASTQ quality control
- 2 Filtering & denoising
- 3 ASV inference with DADA2

04

Ecological Analysis

Connect sequence variants to community ecology



- 1 Taxonomic assignment
- 2 Alpha & beta diversity
- 3 Ecological interpretation

DAY 1 · Raw reads → ASVs

DAY 2 · ASVs → Ecology

Workshop format and expectations

Concept-focused, hands-on, and based on real Jeju ARMS data

01

Workshop purpose

Process metabarcoding data and connect results to ecology

- 1 Process metabarcoding data
- 2 Interpret analytical outputs
- 3 Connect patterns to ecology

02

Learning approach

Focus on concepts, not equations or coding theory

- 1 Why each step is performed
- 2 What each output means
- 3 No coding background required

03

Hands-on Participation

Install R and run the prepared scripts

- 1 Download files from **GitHub**
- 2 Use your own laptop
- 3 Watching only is also fine

04

Teaching Dataset

Reconstructed from a previous Jeju Island ARMS metabarcoding study

- 1 Realistic biological data
- 2 Simplified for the workshop
- 3 Built for ecological analysis

github.com/7LuckySong/jeju-arms-metabarcoding-workshop v2026

Jeju ARMS Case Study

Do different marine habitats support different ARMS-associated communities?

GJ

Gangjeong

Macroalgae-dominated habitat

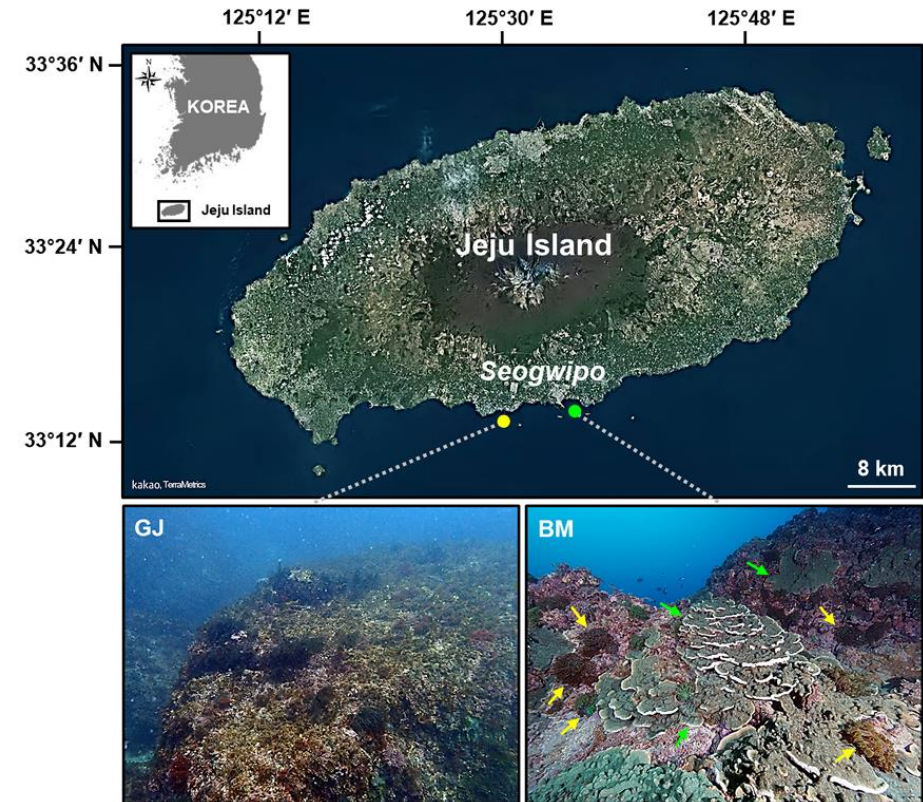
Turf-forming algae
Depth: ~13 m

BM

Bomok

Coral-dominated habitat

Scleractinian corals
Depth: ~12 m



Study sites on the southern coast of Jeju Island

Lee et al. (2024)

CORE
QUESTION

Different surrounding habitats → Different ARMS communities?

Visual survey vs. DNA metabarcoding

Two complementary approaches

Visual survey

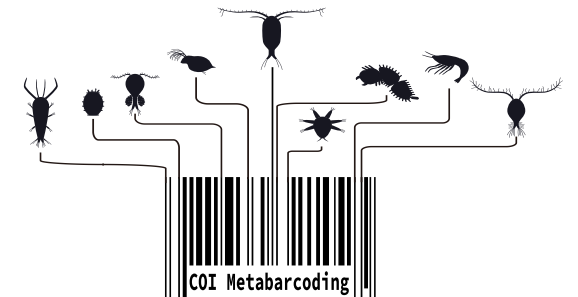
- Morphological identification
- Observer-dependent
- Limited resolution for some taxa



Lee et al. (2024)

DNA metabarcoding

- Higher sensitivity: Detect cryptic organisms
- More reproducible
- Potentially higher taxonomic resolution



Combining visual surveys and DNA metabarcoding provides a more complete view of biodiversity.

01

DNA barcode & NGS sequencing

- What is a DNA barcode?
- COI target region and primers
- PCR product to sequencing library
- Illumina paired-end sequencing

01

What is a DNA barcode?

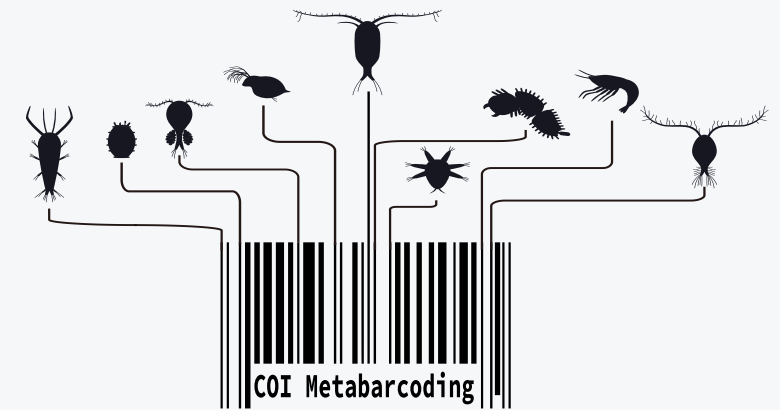
Why do we target specific DNA regions?

Common barcode markers

- 16S rRNA — bacteria and archaea
- 18S rRNA — broad eukaryotic diversity
- COI — animals
- ITS — fungi

A useful DNA barcode should:

- be **amplifiable** across many taxa
- contain **enough sequence variation** to distinguish taxa
- have well-represented **reference sequences**



COI target region and primers

Target marker: mitochondrial cytochrome c oxidase subunit I (COI)

Why COI?

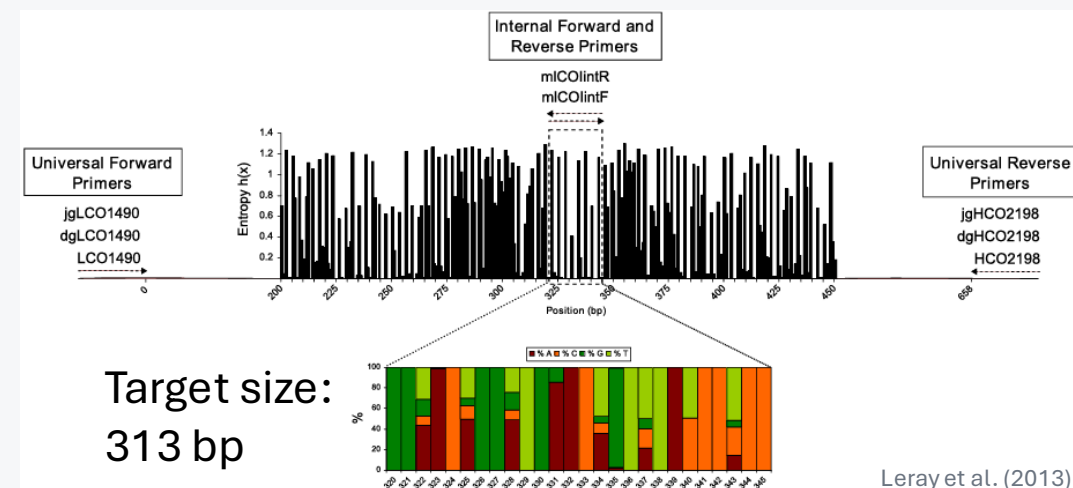
- Widely used as the standard DNA barcode for animals
- Provides high taxonomic resolution for many animal groups
- Well represented in public reference databases

COI barcode region

- Mitochondrial protein-coding gene
- Full COI gene: approximately **1.5 kb**
- Standard animal barcode region: approximately **658 bp**

Primer pair used in this workshop

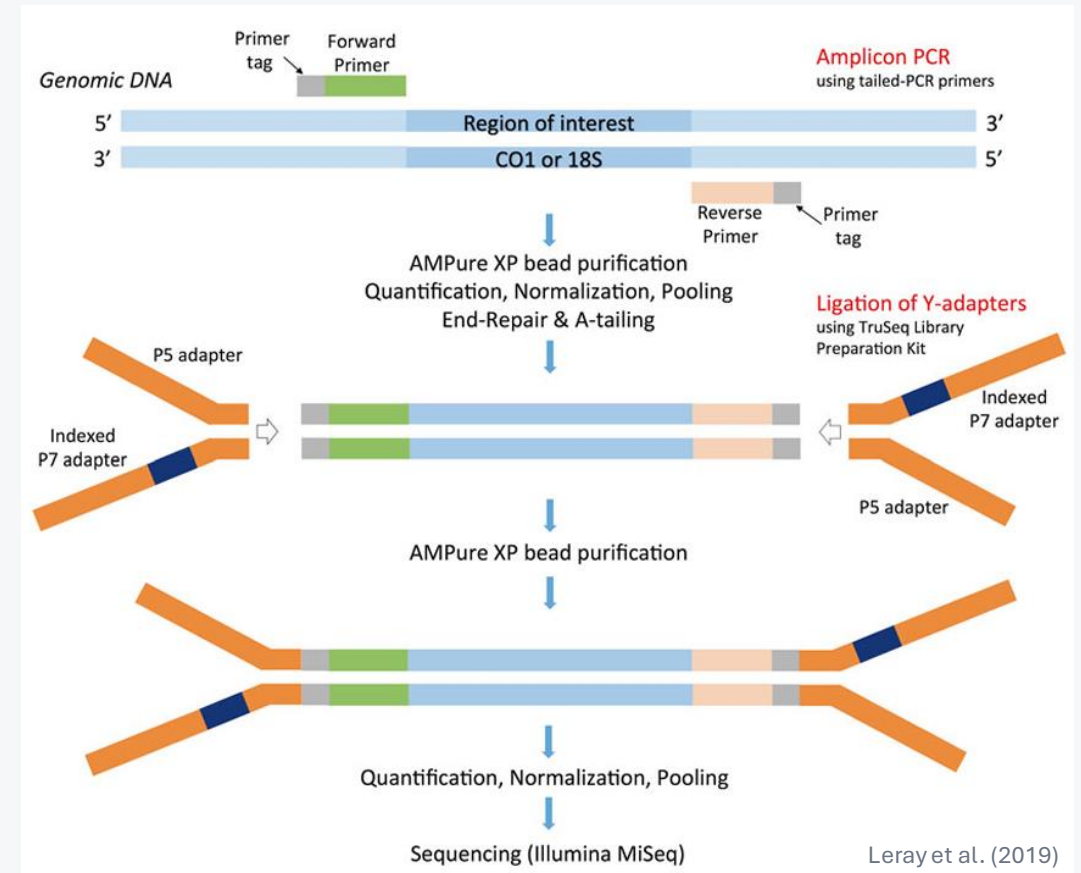
- **mlCOIintF (Forward)**
5' -GGWACWGGWTGAACWGTWTAYCCYCC-3'
- **jgHCO2198 (Reverse)**
5' -TAIACYTCIGGRTGICCRARAAYCA-3'



From PCR product to sequencing library

How do we prepare DNA fragments for Illumina sequencing?

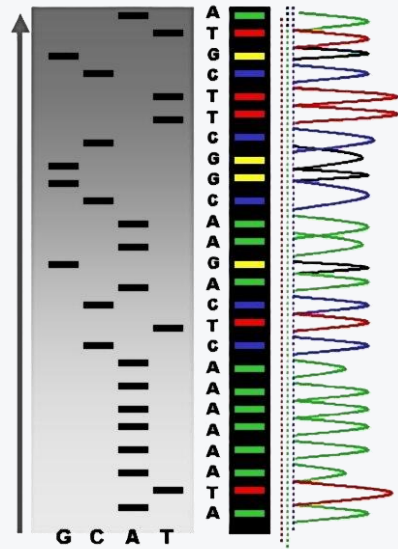
- **PCR:** amplifies the target COI barcode region
- **Adapter:** allows DNA fragments to bind to the Illumina flow cell
- **Index:** identifies the sample of origin after sequencing



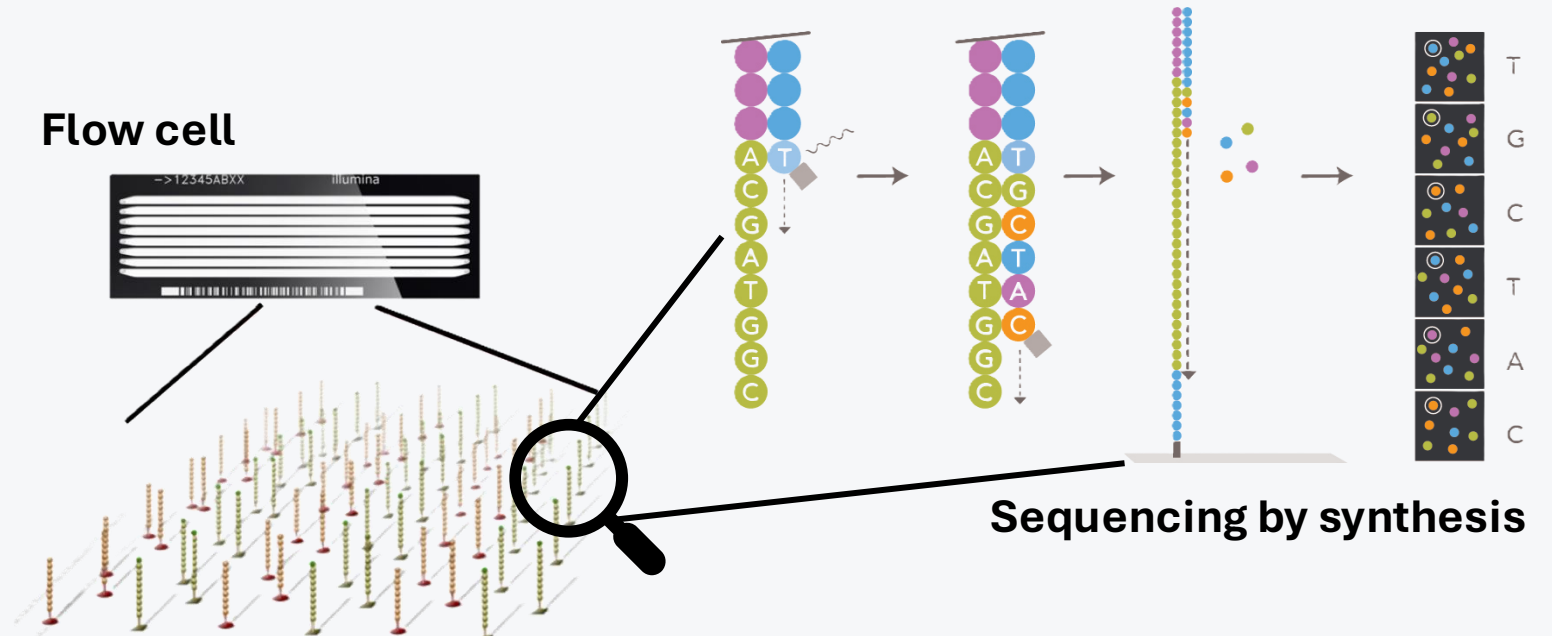
Illumina sequencing

How can millions of DNA fragments be sequenced at once?

- **Sanger sequencing:** sequences individual DNA templates separately
- **Illumina sequencing:** sequences millions of DNA fragments in parallel on a **flow cell**



Sanger sequencing



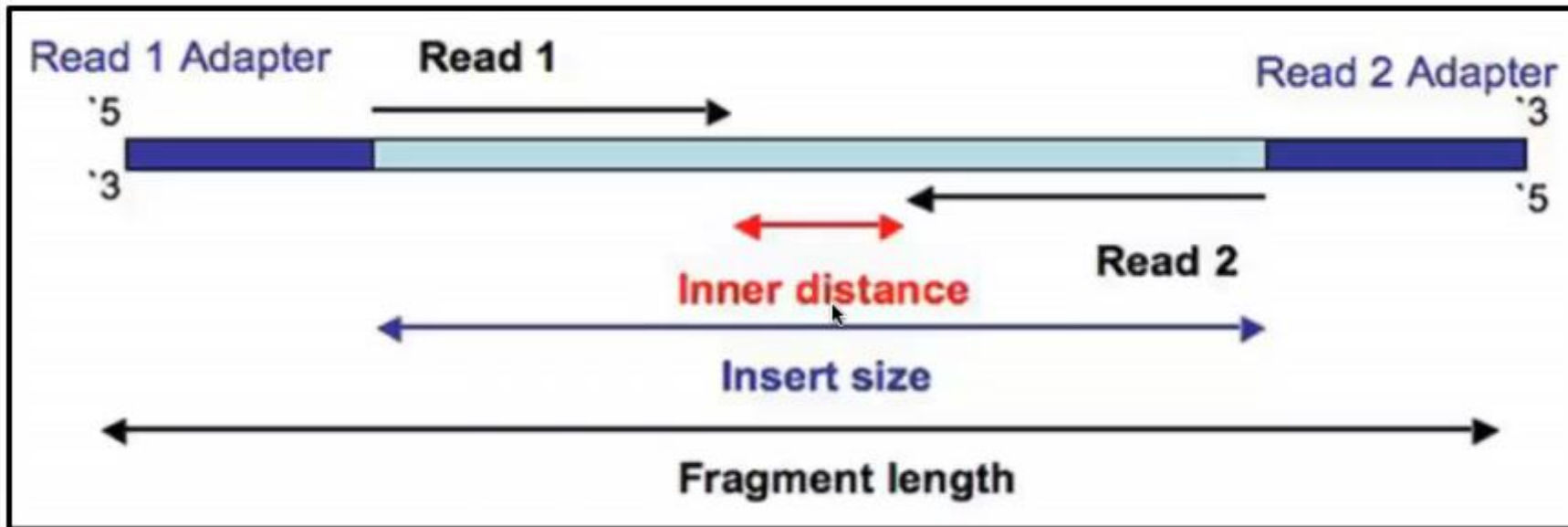
Illumina sequencing

KEY POINT • Illumina sequencing generates millions of reads in parallel.

Illumina sequencing – paired end sequencing

How does paired-end sequencing work?

- Both ends of the same DNA fragment are sequenced.
 - **Read 1 (R1):** forward read
 - **Read 2 (R2):** reverse read
- **Overlapping regions** can be merged into a single sequence.



Paired-end sequencing

02

BIOINFORMATICS PROCESSING

From raw sequencing reads to reliable ASVs

02

Raw sequencing output

What does the sequencing company actually give us?

Raw data statistics

Sample ID	Total read bases (bp)	Total reads	GC (%)	AT (%)	Q20 (%)	Q30 (%)
Bomok1	7,021,126	23,326	39.44	60.57	91.00	71.50
Bomok2	6,162,674	20,474	42.53	57.48	91.50	72.00
Bomok3	7,650,216	25,416	40.04	59.96	92.50	72.50
Bomok4	8,443,050	28,050	41.30	58.71	91.00	71.00
Gangjeong1	6,431,166	21,366	40.59	59.42	91.50	72.50
Gangjeong2	8,019,844	26,644	38.65	61.36	93.00	73.50
Gangjeong3	9,118,494	30,294	41.24	58.77	91.50	72.50
Gangjeong4	9,118,494	30,294	41.24	58.77	91.50	72.50

File contents

Download link	File size	md5sum
Bomok1_R1_001.fastq.gz	2.9 M	fb039b00b39bf46efe3b7db3bed9bf8b
Bomok1_R2_001.fastq.gz	3.2M	2cec4e1eede1a873a4fd24336a307033
Bomok2_R1_001.fastq.gz	2.6M	fad0e3e992afb749e4ffb3bd5dc11204
Bomok2_R2_001.fastq.gz	2.8M	62f0b05d654ce3907e1c5493aae669b6
Bomok3_R1_001.fastq.gz	3.2M	75615eb71d1309c74c8b2124b02a6c60
Bomok3_R2_001.fastq.gz	3.5M	5ffc12abf2dcb8bd7c4cf0a2e077a5ed
Bomok4_R1_001.fastq.gz	3.6M	9e4aeae7426ffe10d190da6c506266b
Bomok4_R2_001.fastq.gz	3.9M	80d7d19053393ea496abf3f0624e2f61
Gangjeong1_R1_001.fastq.gz	2.7M	f9f349e4f5a96202ecb49c88ad0cdf86
Gangjeong1_R2_001.fastq.gz	2.9M	73bcfe986778ec2a6043728ab7a624ed
...		

- Sequencing output

Sample A

→ **R1:** Sample_R1_001.fastq.gz

→ **R2:** Sample_R2_001.fastq.gz

- Sequencing statistics:

- read counts
- total bases
- GC/AT content

- Quality metrics: Q20 and Q30

What is inside a FASTQ file?

How are sequence and quality information stored?

- Each sequencing read is stored as a four-line FASTQ record.

FASTQ format

@SRR001666.1 071112_SLXA-EAS1_s_7:5:1:817:345 length=72 **Field 1**

GGGTGATGGCCGCTGCCGATGGCGTCAAATCCCACCAAGTTACCCTTAACAACCTTAAGGGTTTTCAAATAGA **Field 2**

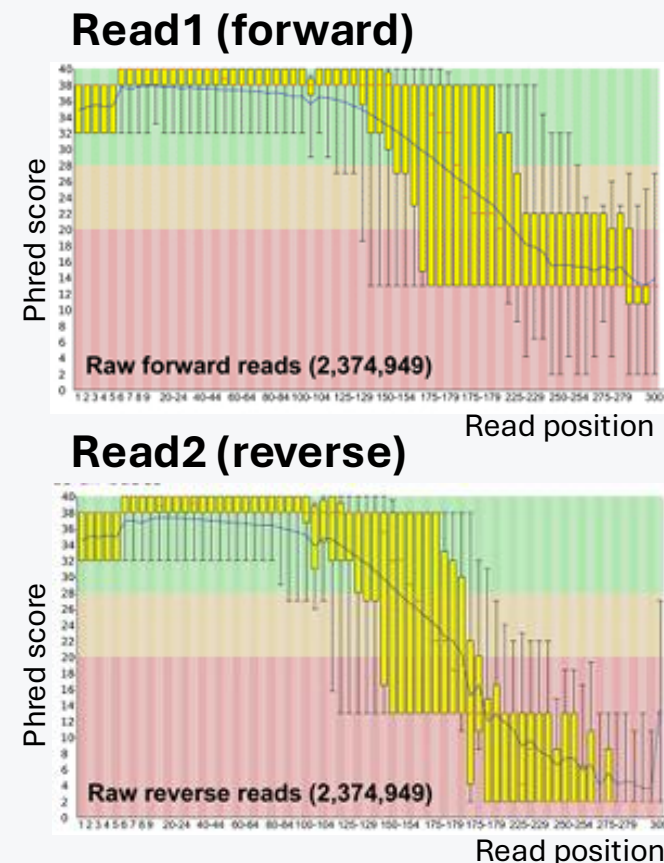
+SRR001666.1 071112_SLXA-EAS1_s_7:5:1:817:345 length=72 **Field 3**

|||||.....|9|G9|C|.....|D|.....>...../ **Field 4**

 Quality scores are encoded as ASCII characters.

Line	Contents
1	@ Read identifier
2	DNA sequence
3	+ Separator
4	Base-quality scores corresponding to each nucleotide in Line 2

- Higher Q = higher confidence in the base call
 - Q20: ~99% base-call accuracy
 - Q30: ~99.9% base-call accuracy
- Modern Illumina FASTQ files commonly use Phred+33 encoding
- Read quality often declines toward the end of a read



- Low-quality bases can create false sequence variants.

03

Practice

R/R studio setup
FASTQ quality control
ASV inference

03

R and RStudio

Why do we use them for metabarcoding analysis?



- **Free & open-source programming language and environment** for statistical computing
- Runs on **Windows, macOS, and Linux**
- Large ecosystem of packages for **bioinformatics, statistics, and ecology**
- Script-based analysis supports **reproducibility**

```
R version 4.6.1 (2026-06-24 ucrt) -- "Happy Hop"
Copyright (C) 2026 The R Foundation for Statistical Computing
Platform: x86_64-w64-mingw32/x64

R은 자유 소프트웨어이며, 어떠한 형태의 재배포도 가능합니다.
또한, 일정한 조건하에서 이것을 재배포 할 수 있습니다.
비밀과 관련된 상세한 내용은 'license()' 또는 'licence()'를 통하여 확인할 수 있습니다.

R은 많은 기여자들이 참여하는 공동프로젝트입니다.
'contributors()'라고 입력하시면 이에 대한 더 많은 정보를 확인할 수 있습니다.
그리고, R 또는 R 패키지들을 출판할 때 인용하는 방법은 'citation()'를 통해 확인하실 수 있습니다.

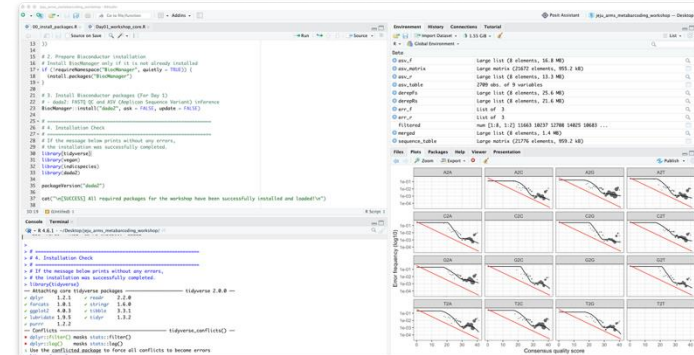
'demo()'를 입력하시면 몇가지 예제를 보실 수 있으며, 'help()'를 입력하시면 온라인 도움말을 이용하실 수 있는
또한, 'help.start()'의 입력을 통하여 HTML로 브라우저에 의한 도움말을 이용하실 수 있습니다.
R의 도움말을 보시려면 'q()'를 입력하십시오.

> R.version.string
[1] "R version 4.6.1 (2026-06-24 ucrt)"
> |
```

R Console



- An **integrated development environment (IDE)** for R
- **Write and run R code** directly from the source editor
- Organizes scripts, files, plots, objects, and help in **one workspace**
- **Projects** help organize analysis files and working directories



RStudio

Installing R on Windows

What do we need before using RStudio?



1

DOWNLOAD

Download R for Windows → base
Click “[Download R-4.6.1 for Windows](#)”

* Note: macOS users should download the **macOS version**.

2

INSTALL

Run the Windows installer (.exe)
Keep the **default installation options**

3

CHECK

Open R and run: `R.version.string`
Confirm that R starts without errors

For this workshop: no PATH setup or Rtools installation is needed.

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- [Download R for Linux \(Debian, Fedora/Redhat, Ubuntu\)](#)
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Click this link: <https://ftp.yz.yamagata-u.ac.jp/pub/cran/>
or search “**R download**” on Google.

> `R.version.string` → Confirm the installed R version

```
> R.version.string  
[1] "R version 4.6.1 (2026-06-24 ucrt) "
```

KEY POINT • Install R first; RStudio will use the existing R installation.

Installing RStudio Desktop

How do we get the interface for working with R?



search “**Rstudio download**” on Google

1

DOWNLOAD

Go to the RStudio download page
Choose “**Download RStudio**” >> click the Download link

2

INSTALL

Run the Windows installer (.exe)
Keep the default installation options

3

LAUNCH

Open RStudio
Check that the Console starts with your installed R version

Important: R must be installed before RStudio. RStudio usually detects the installed R automatically.

Platform	Download
Windows 11	RStudio-2026.08.1-195.exe
Windows 11 (zip archive)	RStudio-2026.08.1-195.zip
macOS 14+	RStudio-2026.08.1-195.dmg
Ubuntu 22 / 24 / 26, Debian 13	rstudio-2026.08.1-195-amd64.deb
Ubuntu / Debian (tar.gz)	rstudio-2026.08.1-195-amd64-debian.tar.gz
RHEL 9 / 10, Fedora	rstudio-2026.08.1-195-x86_64.rpm

[RStudio IDE | The Premier Free IDE for R](#)

Troubleshooting

R should be installed before RStudio.

Meet RStudio

Where do we write code, see results, and manage files?

The screenshot displays the RStudio IDE with the following components:

- Source Pane (1):** Contains R code for locating paired FASTQ files, sorting them, and checking for mismatches. The code includes comments and function calls like `list.files`, `sort`, and `plotQualityProfile`.
- Console (2):** Shows the execution output, including file paths and the result of `plotQualityProfile(fns[1])`. It also displays warning messages about the `serialize` function.
- Environment (3):** Lists the objects created during the analysis, such as `fnRs`, `chr`, `FWD`, `max_ee`, `primer_dir`, `primer_mismatch`, `project_dir`, `raw_dir`, `result_dir`, `REV`, `sample_names`, `sample_names_F`, and `sample_names_R`.
- Plots (4):** Displays a quality score plot for the file `201908_ARMS_KJ_R2_001.fastq.gz`. The y-axis is labeled 'Quality Score' (0 to 40) and the x-axis is labeled 'Cycle' (0 to 300). The plot shows a decreasing trend in quality score over cycles, with a red line indicating the mean and a shaded area representing the standard deviation. A red text label at the bottom left indicates 'Reads: 10683'.

Source

1 Write and save R scripts

Console

2 Run commands and view messages

Environment

3 Inspect objects created during analysis

Files / Plots

4 Manage files, plots, packages, and help

WORKSHOP TIP Run selected code with **Ctrl + Enter**

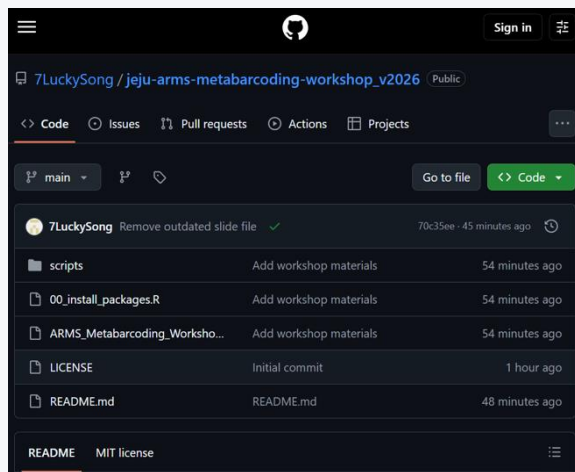
Download the workshop data

Repository → Release v1.0.0 → Download and unzip

01

Open the GitHub repository

Visit github page:

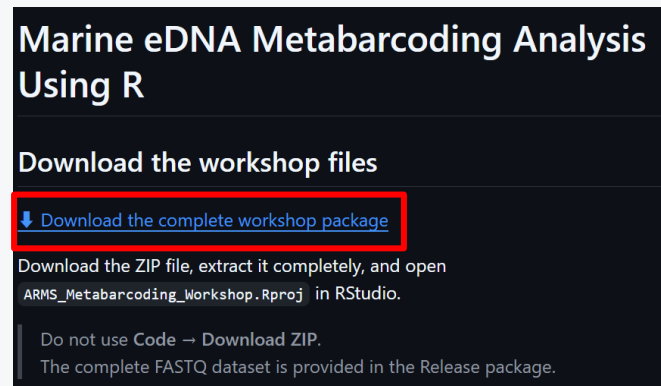


github.com/7LuckySong/jeju-arms-metabarcoding-workshop_v2026

02

Choose Release v1.0.0

Click “Download the complete workshop package”

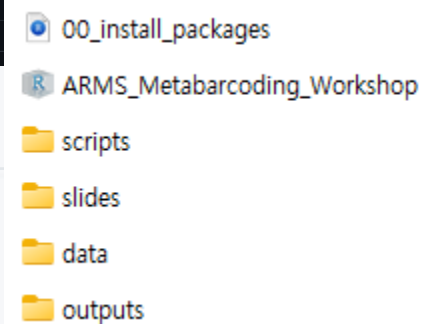
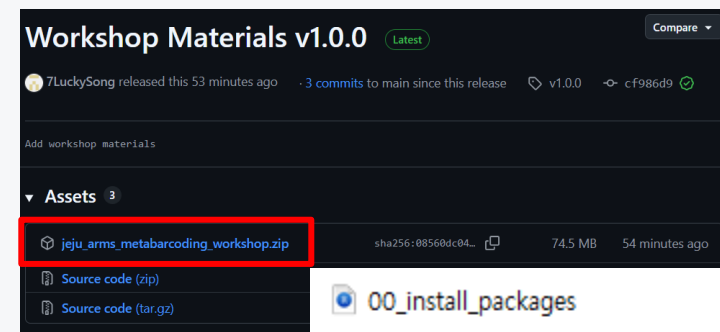


03

Download & unzip the workshop files

Download
“jeju_arms_metabarcoding_workshop.zip”

Unzip the file!



RUN A QUICK CHECK

Load dada2 and check that the data / folder is visible.



ARMS_Metabarcoding_Workshop
R Project
267바이트

Download: ARMS_Metabarcoding_Workshop

1

OPEN THE PROJECT

Open “ARMS_Metabarcoding_Workshop.Rproj” before running the scripts

2

INSTALL THE PACKAGES

- Open the “00_install_packages.R”
- **Run all script code >> automatically installed**

3

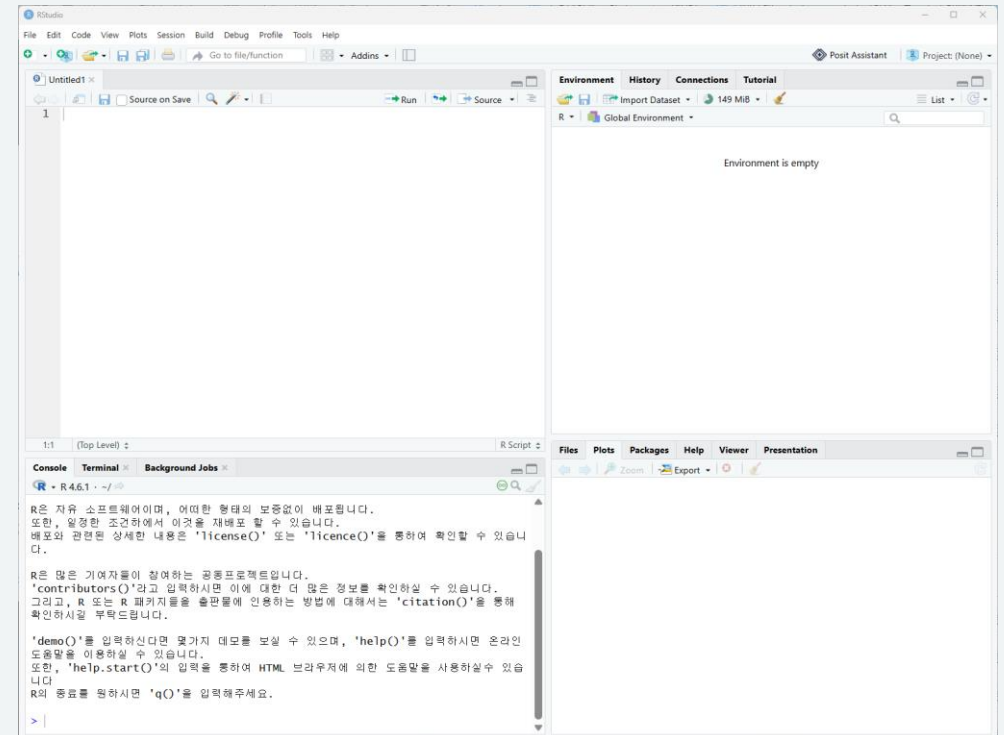
RUN A QUICK CHECK

Load dada2 and confirm that the workshop data folders are visible

PACKAGE SETUP

```
install.packages(c("tidyverse", "vegan", "indicspecies"))
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install("dada2")
library(tidyverse)
library(vegan)
library(indicspecies)
library(dada2)

packageVersion("dada2")
```



```
> packageVersion("dada2")
[1] '1.34.0'
```

Why do raw reads need processing?

Why problems must we solve before inferring ASVs?

RAW SEQUENCING READS

Biological information + technical errors

A T G C C **G** T A A C T G

low Q

A T G C C G T A A C **A** G

error?

A T G **T** C G T A A C T G

Chimeric?

C T A T G C C G T A A C T G

Adapters?

Raw reads are observations
— not yet biological sequence variants.

What must we handle?



Low-quality bases

Trim or filter unreliable positions and reads.



Sequencing / PCR errors

Model technical errors so they are not mistaken for real variants.



Primer / adapter sequence

Remove non-biological sequence before downstream inference.



Chimeric sequences

Identify PCR-derived combinations that do not represent true templates.

Biological information +
technical errors

QUALITY CONTROL
& ERROR MODELING

Biological sequence
variants

ASVs

How are ASVs different from OTUs?

Two different strategies for defining sequence units

OTU Cluster similar sequences

A predefined similarity threshold determines which sequences are grouped together.

Sequence A ATGCCGTAAGTG
 Sequence B ATGCCGTAAC**T**A
 Sequence C ATGCCGTAAC**C**CG

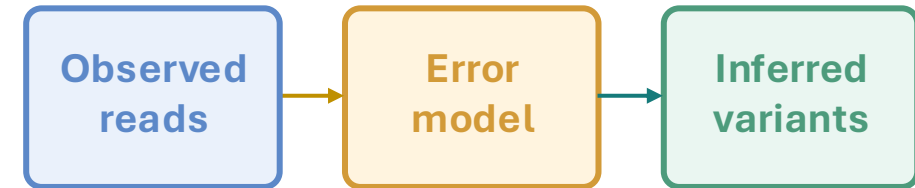
} — OTU 1

Example threshold: 97% similarity

The similarity threshold is chosen by the analyst.

ASV Infer sequence variants

DADA2 models sequencing errors and asks whether observed differences are consistent with error or with distinct variants.

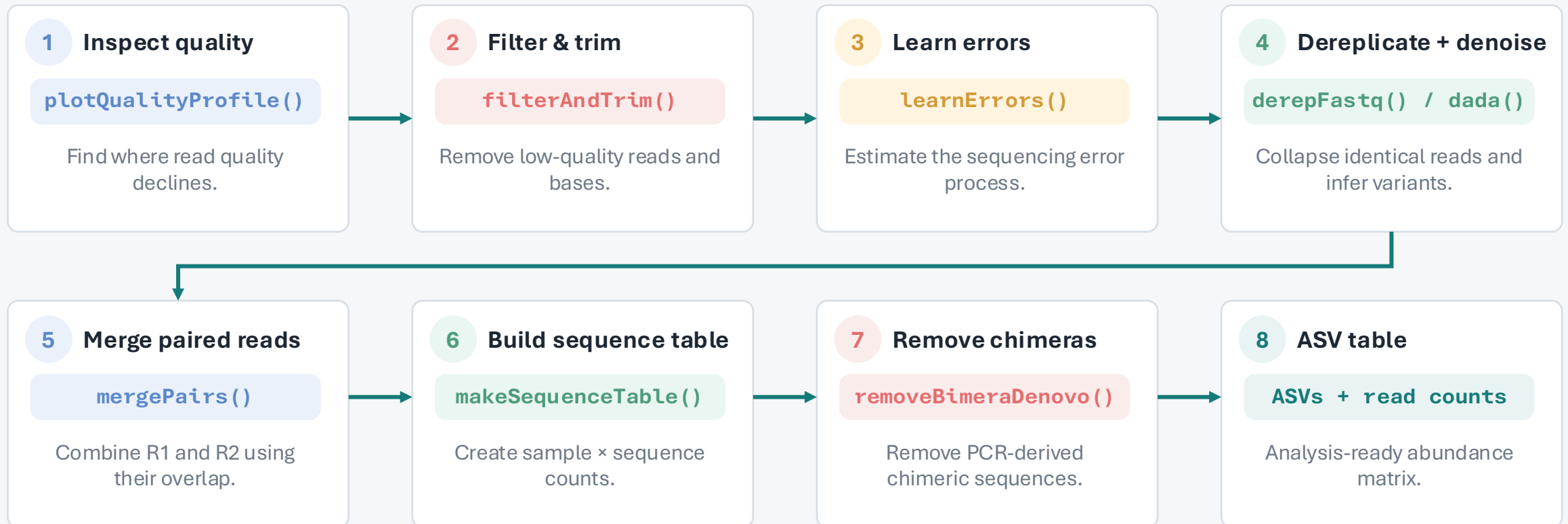


ATGCCGTAAC × 124	ATGCCGTAAC ASV1
ATG T CGTAAC × 3	ATGTCGTAAC ASV2

No arbitrary similarity threshold is used to define ASVs.

DADA2 workflow: from FASTQ to ASV table

How do raw sequencing reads become an analysis-ready sequence table?



TODAY'S FOCUS Understand why each step is performed — you do not need to memorize every function or parameter.

KEY POINT • DADA2 progressively reduces technical noise while preserving biological sequence variation.

Practical 1 — Open the workshop project & settings

Let's make sure everyone starts from the same working environment.

1

OPEN THE PROJECT

Double-click ARMS_Metabarcoding_Workshop.Rproj

2

CHECK THE FOLDERS

Confirm data/ outputs/ scripts/ slides/

3

OPEN THE SCRIPT

Open scripts/Day01_workshop_core.R

4

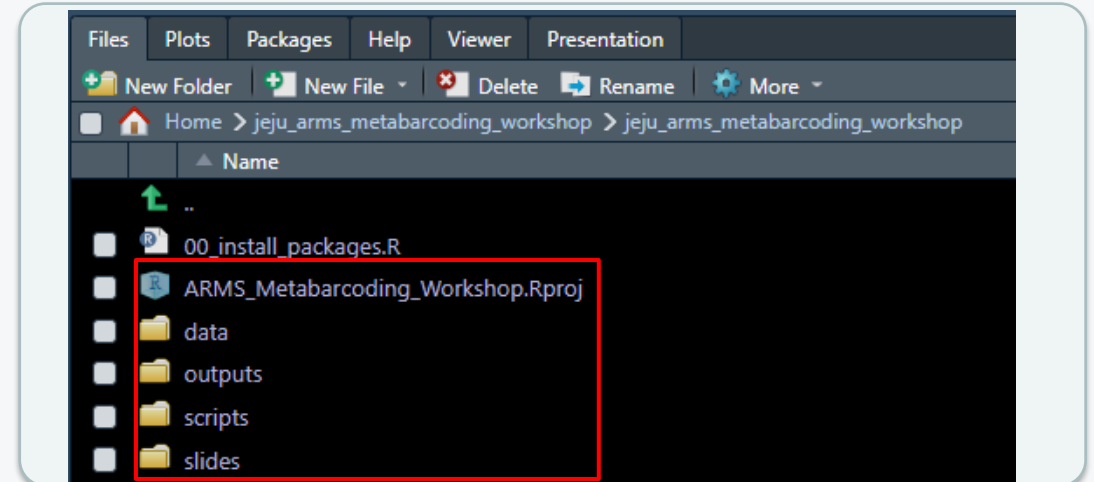
RUN THE FIRST CHECK

Run selected code with Ctrl + Enter

QUICK CHECK

Expected: R1/R2 FASTQ files

```
library(dada2)
list.files("data/fastq")
```



When the setup is correct

```
> list.files("data/fastq")
[1] "Bomok1_R1_001.fastq.gz"  "Bomok1_R2_001.fastq.gz"
[3] "Bomok2_R1_001.fastq.gz"  "Bomok2_R2_001.fastq.gz"
[5] "Bomok3_R1_001.fastq.gz"  "Bomok3_R2_001.fastq.gz"
[7] "Bomok4_R1_001.fastq.gz"  "Bomok4_R2_001.fastq.gz"
[9] "Gangjeong1_R1_001.fastq.gz" "Gangjeong1_R2_001.fastq.gz"
[11] "Gangjeong2_R1_001.fastq.gz" "Gangjeong2_R2_001.fastq.gz"
[13] "Gangjeong3_R1_001.fastq.gz" "Gangjeong3_R2_001.fastq.gz"
[15] "Gangjeong4_R1_001.fastq.gz" "Gangjeong4_R2_001.fastq.gz"
```

KEY POINT ·

Open the RStudio Project first and confirm the input files before starting the analysis.

Practical 2 — Inspect read quality

Where should we trim without losing paired-end overlap?

QUALITY

FILTER

ERROR

DENOISE

MERGE

CHIMERA

ASV TABLE

What are we looking for?

- Compare quality patterns between R1 and R2
- Find where quality declines near the read ends
- Choose trimming positions for the next step

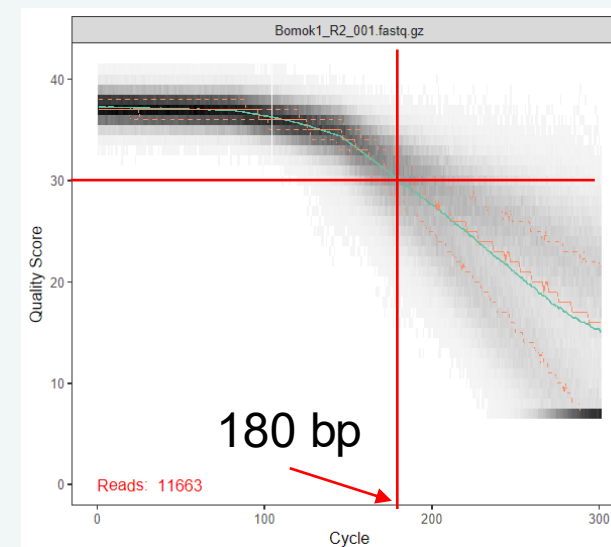
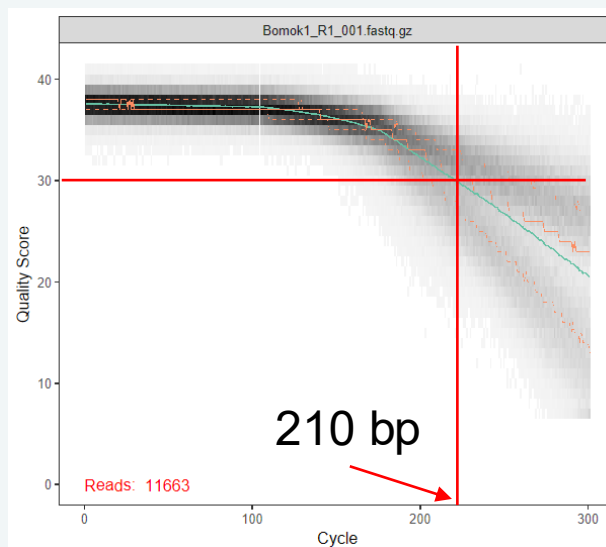
R CODE

```
# 2. Inspect Read Quality (Raw) -----
plotQualityProfile(fwd[1:2])
plotQualityProfile(rev[1:2])
```

Do not trim by quality alone

Forward and reverse reads **must still overlap** after trimming.

Keep enough high-quality sequence for `mergePairs()`.



ASK THE CLASS

1. Which read loses quality earlier?
2. Where would you truncate R1 and R2?
3. Will enough overlap remain?

Practical 3-5 — Filter and learn errors

How do we remove low-quality reads and model sequencing errors?

QUALITY

FILTER**ERROR**

DENOISE

MERGE

CHIMERA

ASV TABLE

filterAndTrim()

Remove low-quality reads/bases using the trimming and filtering settings selected from the quality profiles.

learnErrors()

Learn the sequencing error pattern directly from the filtered reads.

WORKSHOP CODE

```
# 3. Filter and Trim -----
primer_len <- c(26, 26) # COI FWD and REV primer lengths
trunc_len <- c(210, 180)

filt_f <- file.path(filt_path, paste0(sample_id, "_F.fastq.gz"))
filt_r <- file.path(filt_path, paste0(sample_id, "_R.fastq.gz"))

filtered <- filterAndTrim(
  fwd, filt_f, rev, filt_r, truncLen = trunc_len, trimLeft = primer_len,
  maxN = 0, maxEE = c(2, 4), truncQ = 2, rm.phix = TRUE, matchIDs = TRUE,
  compress = TRUE, multithread = FALSE)

# 4. Compare Before & After Trimming -----
plotQualityProfile(c(fwd[1], filt_f[1], rev[1], filt_r[1]))

# 5. Learn error rates -----
err_f <- learnErrors(filt_f)
err_r <- learnErrors(filt_r)

plotErrors(err_f, nominalQ = TRUE); plotErrors(err_r, nominalQ = TRUE)
```

Practical 3-5 — Filtering results and learned error model

What changed after filtering, and what error pattern did DADA2 learn?

QUALITY

FILTER

ERROR

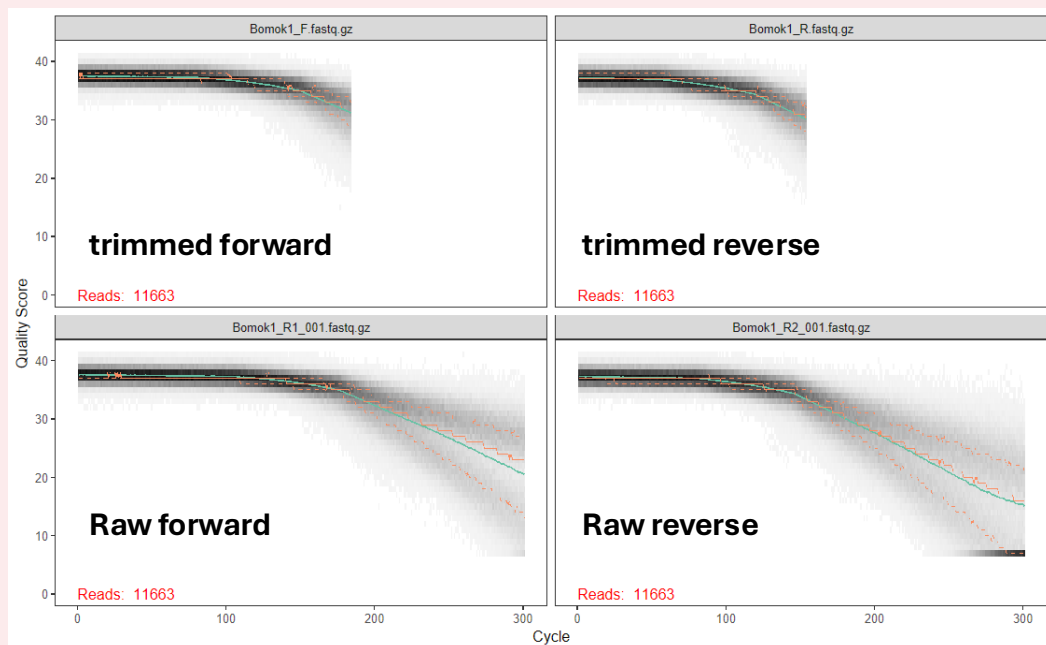
DENOISE

MERGE

CHIMERA

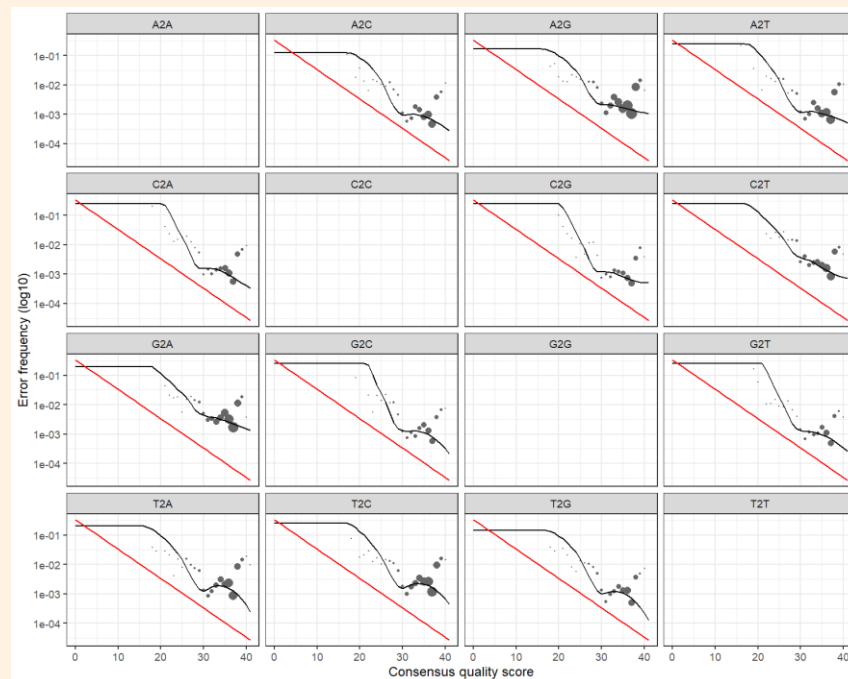
ASV TABLE

`filterAndTrim()`



Low-quality tails are trimmed, and poor-quality reads are removed before denoising.

`learnErrors()`



DADA2 estimates how error rates change with quality score using the filtered data.

Practical 6 — Dereplication and denoising

How does DADA2 separate sequencing errors from real sequence variants?

QUALITY

FILTER

ERROR

DENOISE

MERGE

CHIMERA

ASV TABLE

derepFastq()

Collapse identical reads into unique sequences and record how many times each sequence occurs.

dada()

Use the learned error model to infer sequence variants separately from the forward and reverse reads.

WORKSHOP CODE

```
# 6. Dereplication and ASV inference (Denoise) -----  
# 1) Dereplication  
derepFs <- derepFastq(filt_f)  
derepRs <- derepFastq(filt_r)  
  
# 2) ASV inference  
asv_f <- dada(derepFs, err = err_f)  
asv_r <- dada(derepRs, err = err_r)
```

Practical 7-9 — Merge reads, remove chimeras, and build the ASV table

How do denoised paired reads become the final ASV table?

QUALITY

FILTER

ERROR

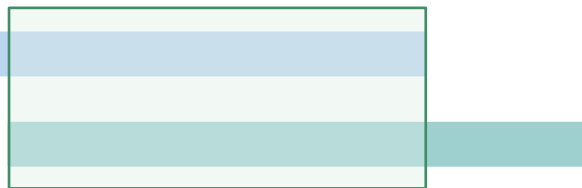
DENOISE

MERGE**CHIMERA****ASV TABLE**

1 · Merge paired reads

Forward (R1)

Reverse (R2)



overlap

mergePairs()

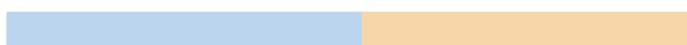
Denoised R1 and R2 must overlap and agree in the shared region.

2 · Remove PCR chimeras

Parent A

Parent B

Chimera



removeBimeraDenovo()

Removes PCR artifacts formed by combining parts of different parent sequences.

FINAL PROCESSING

```
# 7. Merge paired reads -----
min_overlap <- 20

merged <- mergePairs(asv_f, derepFs, asv_r, derepRs,
                     minOverlap = min_overlap, maxMismatch = 0)

names(merged) <- sample_id
sequence_table <- makeSequenceTable(merged)

# 8. Remove chimeras -----
asv_matrix <- removeBimeraDenovo(
  sequence_table, method = "consensus", multithread = threads)

# 9. Export ASV table and representative sequences -----
abundance_order <- order(colSums(asv_matrix), decreasing = TRUE)
asv_matrix <- asv_matrix[, abundance_order, drop = FALSE]
...
```

KEY POINT · Denoised reads are merged, PCR chimeras are removed, and the remaining sequences form the final ASV table.

What did DADA2 give us?

Output 1 — ASV abundance table

ASV × sample count matrix

Each value is the number of reads assigned to an ASV in a sample.

	Sample 1	Sample 2	Sample 3
ASV1	1250	21	0
ASV2	32	845	102
ASV3	0	15	520
...	...	...	...

ASV table This table is the community matrix used for downstream ecological analysis.

How to read it

- ROW

ASV
one inferred sequence variant
- COLUMN

Sample
one biological sample
- VALUE

Read count
abundance of that ASV
in the sample

Day 2 input: community abundance data

What did DADA2 give us?

Output 2 — Representative ASV sequences

Representative sequences (FASTA)

One nucleotide sequence is retained for each ASV.

```
>ASV1  
ATGGCACCTTTGACT...
```

```
>ASV2  
ATAGCTCGTACCAA...
```

```
>ASV3  
TTGCCGATGGTCAA...
```

Sequence identity only — no organism name yet

Do we know which organism ASV1 is?

Not yet.

DADA2 infers distinct sequence variants,
but it does not assign taxonomy.

**Sequence inference
≠ Taxonomic identification**

Day 1 — What did we accomplish?

From ARMS samples to an ASV dataset

Today's workflow

Illumina Paired-end sequencing

FASTQ Raw reads + quality

QC + DADA2 Error-aware
sequence inference

ASV outputs Table + sequences

Take-home messages

1

Raw reads contain technical errors

Quality control is required before treating sequence differences as biological variation.

2

DADA2 uses an error model

It learns sequencing-error patterns and infers ASVs rather than clustering at an arbitrary similarity threshold.

3

Day 1 ends with two key outputs

The ASV abundance table and representative ASV sequences become the starting point for taxonomy and ecology.

Overview — workshop workflow

From a marine sample to ecological interpretation

01

Sampling & DNA Preparation

Collect and prepare the biological material

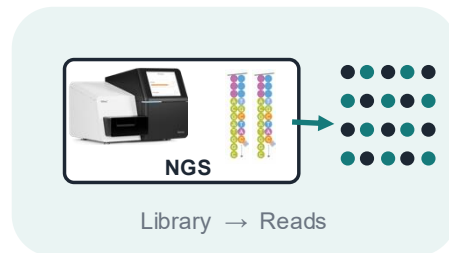


- 1 Marine sample collection
- 2 DNA extraction
- 3 COI barcode amplification

02

NGS Sequencing

Convert amplified DNA into sequencing reads



- 1 Library preparation
- 2 Illumina sequencing
- 3 Raw sequencing output

03

Bioinformatics Processing

Turn raw reads into reliable sequence variants



- 1 FASTQ quality control
- 2 Filtering & denoising
- 3 ASV inference with DADA2

04

Ecological Analysis

Connect sequence variants to community ecology



- 1 Taxonomic assignment
- 2 Alpha & beta diversity
- 3 Ecological interpretation

DAY 1 · Raw reads → ASVs

DAY 2 · ASVs → Ecology

Day 1 answered “What sequence variants are present?” — Day 2 asks “What organisms are they, and how do communities differ?”